Reflexiones individuales

MediAgenda · Hito 2 · Equipo 05

Benjamín Concepción · Dana Wing · Alvaro Guardia · Edwin Vergara

Reflexión individual – Benjamín Concepción

Proyecto: MediAgenda · Hito 2 · Equipo 05

1. ¿Qué hice?

- Migré las páginas de HTML a PHP renombrándolas con `git mv` (index, login, registro, recuperar, las tres `principal_*`, `secundaria_formulario`, error y legal).
- Cambié el encabezado y el pie repetidos por `require_once` de `includes/`, y las filas escritas a mano por `foreach` sobre `config/datos_demo.php`.

2. ¿Qué aprendí?

- En este hito nada se guarda: si reservas una cita, al recargar no existe (el código siempre es CIT-0218).
- Lo que hay son datos de ejemplo en arreglos de PHP (`config/datos_demo.php`) que se leen, y `$_SESSION`, que recuerda datos entre páginas (rol, errores, lo escrito) mientras dura la sesión.

3. Dificultades

Tuve un conflicto al hacer un commit con un cambio necesario para validar el funcionamiento. Dana había añadido un comentario en la línea del `require_once` y el listado de 5 citas, y yo había añadido `exigir_rol(['admin'])` justo debajo de esa misma línea.

Me di cuenta porque GitHub mostró «This branch has conflicts that must be resolved» en el Pull Request. Hice `git merge origin/entrega-hito2` en mi rama y leí las marcas `<<<<<<<`, `=======` y `>>>>>>>`. En el camino, Git respondía «not a git repository» porque la terminal estaba en `C:\repos` y no en la carpeta del proyecto.

Lo resolví conservando las dos cosas (la línea de Dana y mi `exigir_rol`), revisando con `php -l` que no quedaran marcas y probando que el administrador veía el panel y los demás roles no. Así aprendí a corregir un conflicto de este tipo.

4. ¿Qué mejoraría?

[Pendiente de completar por Benjamín: cómo evitaría un conflicto así y qué cambiaría en la forma de trabajar en equipo.]

5. Uso de IA

Usé Claude para consultas y mejoras de las tareas realizadas.

Reflexión individual – Dana Wing

Proyecto: MediAgenda · Hito 2 · Equipo 05

1. ¿Qué hice?

Desarrollé los scripts de procesamiento del backend (`procesar_registro.php`, `procesar_login.php` y `procesar_principal.php`), donde implementé la regla de negocio de máximo 8 citas por día y la validación de solapamiento de horarios. También integré las páginas `panel.php` y `perfil.php` con los includes del equipo. Finalmente, ejecuté los casos de prueba P-07 a P-12 y P-19, guardé las capturas de evidencia y completé mi sección en el plan de pruebas.

2. ¿Qué aprendí?

Aprendí la importancia de construir «esqueletos» seguros desde el principio, usando la constante `APP_ACCESO` y validando el método POST, incluso antes de tener los datos de mis compañeros. También entendí cómo funciona `password_hash` para proteger las contraseñas y cómo Git maneja las ramas, especialmente al hacer un merge desde la rama de entrega sin que se pierdan los cambios locales.

3. Dificultades

Al principio me bloqueé porque intenté escribir toda la lógica de `procesar_principal.php` sin tener aún el archivo `datos_demo.php` de Edwin, lo que me generaba errores de variables no definidas. Tuve que cambiar mi estrategia y hacer primero los esqueletos protegidos. También tuve un susto con Git cuando intenté fusionar la rama `entrega-hito2` y se me abrió el editor vi en la terminal sin saber cómo salir, pero al final logré resolverlo y hacer el commit.

4. ¿Qué mejoraría?

Haría los merge de la rama de entrega a mi rama personal con más frecuencia durante la semana, en lugar de dejarlo todo para el final. Esto ayudaría a detectar conflictos de código a tiempo y evitar el estrés de resolverlos en la terminal justo antes de la entrega.

5. Uso de IA

Usé Qwen como apoyo para entender errores de Git (como el problema con el editor vi), revisar la ortografía de este texto y estructurar la lógica de los esqueletos de PHP antes de integrar las funciones de mis compañeros.

Reflexión individual – Alvaro Guardia

Proyecto: MediAgenda · Hito 2 · Equipo 05

1. ¿Qué hice?

Construí el esqueleto modular del sistema creando la carpeta `includes/` con los archivos `encabezado.php`, `menu.php`, `pie.php` y `funciones.php`. Dentro de `funciones.php` implementé las 6 funciones propias requeridas por la rúbrica: `limpiar()`, `redirigir()`, `mostrar_errores()`, `formatear_fecha()`, `generar_token_csrf()` y `verificar_token_csrf()`, asegurando que todas usaran la constante `APP_ACCESO` definida por el líder del equipo.

También agregué al archivo `css/estilos.css` las clases `.eq05-estado--pendiente` y `.eq05-estado--confirmada`, usando las variables CSS del sistema (`--eq05-aviso-50`, `--eq05-exito-700`) para mantener la consistencia visual con los demás estados. Finalmente, ejecuté manualmente los casos de prueba P-13 a P-18, que validan el comportamiento del servidor ante datos inválidos, caracteres especiales, feriados y ausencias de médicos, y documenté los resultados con sus capturas en el plan de pruebas.

2. ¿Qué aprendí?

Aprendí que en un equipo la arquitectura es responsabilidad compartida: si mis `includes/` fallan, usan nombres distintos o no respetan la convención acordada, bloqueo el trabajo de Dana, Edwin y Benjamín. Comprendí la diferencia real entre la validación del cliente y la del servidor: en las pruebas P-13 y P-14 comprobé que, si se quita el atributo `required` o se modifica el `value` de un `<select>` con las herramientas del navegador, el servidor es la única barrera que impide que entren datos basura a la aplicación.

También entendí que Git no es solo `git add` y `git push`: trabajar con ramas paralelas (`alvaro/hito2` y `entrega-hito2`) y resolver conflictos de Pull Request me obligó a usar comandos como `git rm --cached`, `git restore` y `git fetch` con un propósito real, no solo teórico.

3. Dificultades

Mi mayor problema fue la coordinación con el equipo y la gestión de ramas. Tuve que borrar archivos que yo mismo había creado (`datos_demo.php` y `Producto.php`) porque, al revisar la matriz de responsabilidades, me di cuenta de que invadían el trabajo de Edwin. También tuve que reescribir todos mis archivos porque al principio usé las constantes que sugería una fuente externa (`ACCESO_QZ7`, `sanear_zq()`) en lugar de las que definió el líder del equipo (`APP_ACCESO`, `limpiar()`). Por eso Benjamín rechazó mi Pull Request varias veces.

Otro problema fue que, al migrar `index.html` a `index.php`, mi archivo quedó con el `<main>` vacío y eso rompía la página de inicio, así que tuve que sacarlo del PR para que Benjamín pudiera hacer el merge sin conflictos. Resolver esto me tomó tiempo y me enseñó a leer con atención la matriz antes de escribir código.

4. ¿Qué mejoraría?

Definiría las convenciones de nombres, la estructura de carpetas y la matriz de responsabilidades antes de escribir la primera línea de código, porque perdí tiempo reescribiendo y eliminando archivos por no alinear

los criterios al inicio del hito. También haría los merge desde `entrega-hito2` a mi rama personal con más frecuencia durante la semana, en lugar de dejarlo para el último día. Además, propondría al equipo una revisión cruzada de los Pull Request antes de aprobarlos, para que errores como usar el nombre de constante equivocado se detecten antes.

5. Uso de IA

Usé Qwen como apoyo para generar la estructura inicial de los archivos PHP, depurar comandos de Git cuando la terminal mostraba errores y estructurar los pasos de los casos de prueba P-13 a P-18. También me ayudó a redactar esta reflexión y a organizar los textos del plan de pruebas. Las pruebas las ejecuté yo manualmente en el servidor local (`php -S localhost:8000`), revisé que el código final respetara la lógica del equipo antes de cada commit y verifiqué personalmente que cada captura reflejara el resultado real del sistema.

Reflexión individual – Edwin Vergara

Proyecto: MediAgenda · Hito 2 · Equipo 05

1. ¿Qué hice?

Preparé `config/datos_demo.php` con los datos de prueba del sistema, incluidas las tres citas semilla del profesor y el límite configurable de 8 citas por día (`MAX_CITAS_DIA`). También ejecuté los casos de prueba P-01 a P-06, guardé las capturas como evidencia y completé la matriz de trazabilidad.

2. ¿Qué aprendí?

Aprendí a trabajar con ramas en Git: cada uno sube a la suya y la entrega va en `entrega-hito2`. También entendí por qué hay que validar en el servidor y no solo en el navegador, y cómo `htmlspecialchars()` y `password_hash()` protegen los datos.

3. Dificultades

Tuve errores de Git al configurar mi usuario y al trabajar en dos computadoras, porque un `push` no se había subido. Ahora reviso que no queden cambios pendientes antes de cerrar. En P-05 salieron 12 errores en vez de 13; lo anoté como salió, lo consulté con el equipo y se confirmó que el campo Sexo es opcional.

4. ¿Qué mejoraría?

Agregaría pruebas para los requerimientos que todavía no tienen una y actualizaría la matriz de responsabilidades cuando el equipo cambia de plan.

5. Uso de IA

Usé Microsoft Copilot como apoyo para entender errores de Git y PHP, preparar la guía de pruebas y organizar este texto. Las pruebas las ejecuté yo y revisé el contenido antes de entregarlo.